DESIGN PATTERNS

A Deep-Dive Field Guide

Definitions, UML Structures & Hands-On Java Implementations

Expanded from the original Capgemini "Design Patterns" training deck — every original definition, UML relationship, and code sample preserved and elaborated.

People matter, results count.

Table of Contents

1. Design Patterns — Overview 3

1.1 Design Patterns Usage 3

1.2 Types of Design Patterns 3

2. Creational Patterns 4

2.1 Factory Pattern 4

2.2 Abstract Factory Pattern 6

2.3 Singleton Pattern 7

2.4 Builder Pattern 9

2.5 Prototype Pattern 11

3. Structural Patterns 14

3.1 Adapter Pattern 14

3.2 Bridge Pattern 17

3.3 Composite Pattern 19

3.4 Decorator Pattern 20

3.5 Facade Pattern 22

3.6 Flyweight Pattern 24

3.7 Proxy Pattern 26

4. Behavioral Patterns 29

4.1 Chain of Responsibility Pattern 29

4.2 Command Pattern 31

4.3 Iterator Pattern 33

4.4 Template Method Pattern 35

5. References 37

1. Design Patterns — Overview

A design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. It is a description or template for how to solve a problem that can be used in many different situations. Patterns are formalized best practices that the programmer can use to solve common problems when designing an application or system.

> **Deep dive:** Design patterns are not finished code you copy-paste — they are proven shapes for a solution. Think of them the way an architect thinks of "load-bearing wall" or "cantilever": the pattern names a structural idea so that two engineers instantly understand the trade-offs being made, without re-deriving them from scratch every time.

Patterns were popularised by the "Gang of Four" (Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides) in their 1994 book Design Patterns: Elements of Reusable Object-Oriented Software. That catalogue of 23 patterns, grouped into Creational, Structural, and Behavioral families, is the backbone of this guide.

1.1 Design Patterns Usage

Design patterns have two main usages in software development.

Common platform for developers

Design patterns provide a standard terminology and are specific to a particular scenario. When one engineer says "let's put a Facade in front of this subsystem," the whole team immediately pictures the same shape — no whiteboard required.

Best practices

Design patterns have evolved over a long period of time and they provide best solutions to certain problems faced during software development. Learning these patterns helps inexperienced developers learn software design in an easy and faster way.

> **Why it matters in practice:** Patterns compress decades of trial-and-error into a name and a shape. Skipping them doesn't mean avoiding complexity — it usually means reinventing the same solution poorly, without the vocabulary to discuss it with your team.

1.2 Types of Design Patterns

Creational Patterns

These design patterns provide a way to create objects while hiding the creation logic, rather than instantiating objects directly using the new operator. This gives the program more flexibility in deciding which objects need to be created for a given use case.

Structural Patterns

These design patterns concern class and object composition. The concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities.

Behavioral Patterns

These design patterns are specifically concerned with communication between objects.

2. Creational Patterns

Creational patterns abstract the instantiation process. They help make a system independent of how its objects are created, composed, and represented — so "what gets built" can change without rewriting the code that asks for it.

2.1 Factory Pattern

> *Create objects without exposing the creation logic to the client, and refer to the newly created object using a common interface.*

Definition

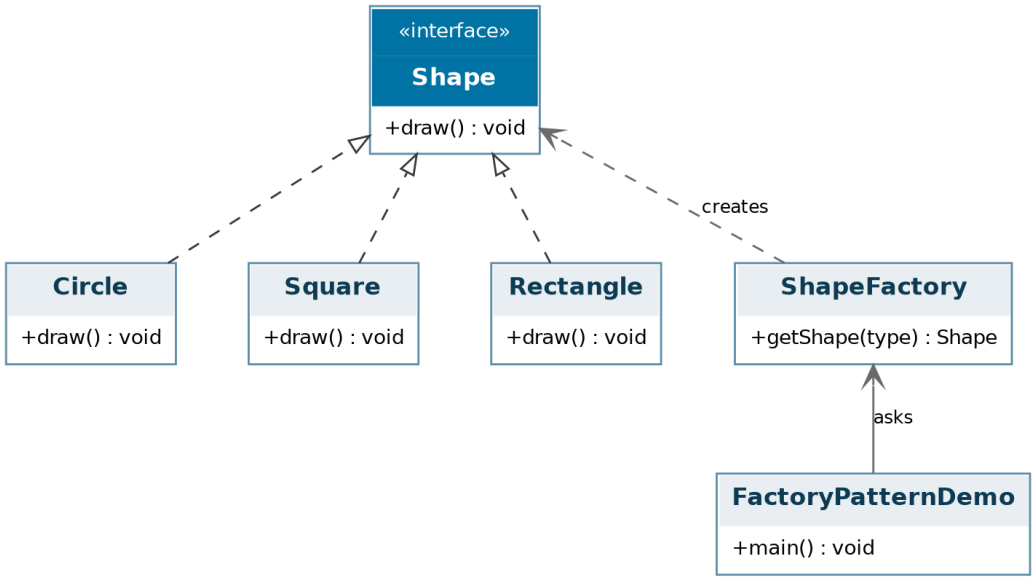
In the Factory pattern, an object is created without exposing the creation logic to the client, and the client refers to the newly created object using a common interface.

> **Real-world analogy:** A pizza counter: you order "a medium pizza," not "combine dough, sauce, cheese, and heat to 250°C for 12 minutes." The counter (factory) decides how to build it; you only see the finished Pizza.

When to reach for it

- A class can't anticipate which concrete subclass it needs to create ahead of time.
- You want to centralize object-creation logic instead of scattering `new` calls across the codebase.
- You want callers to depend only on an interface, not a concrete class.

UML structure



UML Element	Factory Pattern
Shape «interface»	Declares draw(): void. Implemented by every concrete shape.
Circle / Square / Rectangle	Concrete classes; each implements draw().
ShapeFactory	Has getShape(String): Shape — decides which concrete class to instantiate and return.
FactoryPatternDemo	Client; asks ShapeFactory for shapes and calls draw() through the Shape interface.

Hands-on: Java implementation

```
public class FactoryPatternDemo {

  public static void main(String[] args) {
    ShapeFactory shapeFactory = new ShapeFactory();

    //get an object of Circle and call its draw method.
    Shape shape1 = shapeFactory.getShape("CIRCLE");
    shape1.draw();

    //get an object of Rectangle and call its draw method.
    Shape shape2 = shapeFactory.getShape("RECTANGLE");
    shape2.draw();

    //get an object of Square and call its draw method.
    Shape shape3 = shapeFactory.getShape("SQUARE");
    shape3.draw();
  }
}

public class ShapeFactory {

  //use getShape method to get object of type shape
  public Shape getShape(String shapeType){
    if(shapeType == null){
      return null;
    }
    if(shapeType.equalsIgnoreCase("CIRCLE")){
      return new Circle();
    } else if(shapeType.equalsIgnoreCase("RECTANGLE")){
      return new Rectangle();
    } else if(shapeType.equalsIgnoreCase("SQUARE")){
      return new Square();
    }
    return null;
  }
}
```

Walking through the code

- The client never writes `new Circle()` or `new Square()` directly — it asks `shapeFactory.getShape("CIRCLE")`.
- `ShapeFactory` is the only place that knows the mapping from a string key to a concrete class.
- Adding a new shape (e.g. Triangle) only touches ShapeFactory — client code is untouched.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Decouples client code from concrete classes. • Centralizes and simplifies object creation logic. • New product types can be added with minimal ripple effect.	• Adds an extra class/indirection layer for what could be a simple constructor call. • The factory itself can grow into a large if/else or switch block over time.

2.2 Abstract Factory Pattern

> *A super-factory that creates other factories.*

Definition

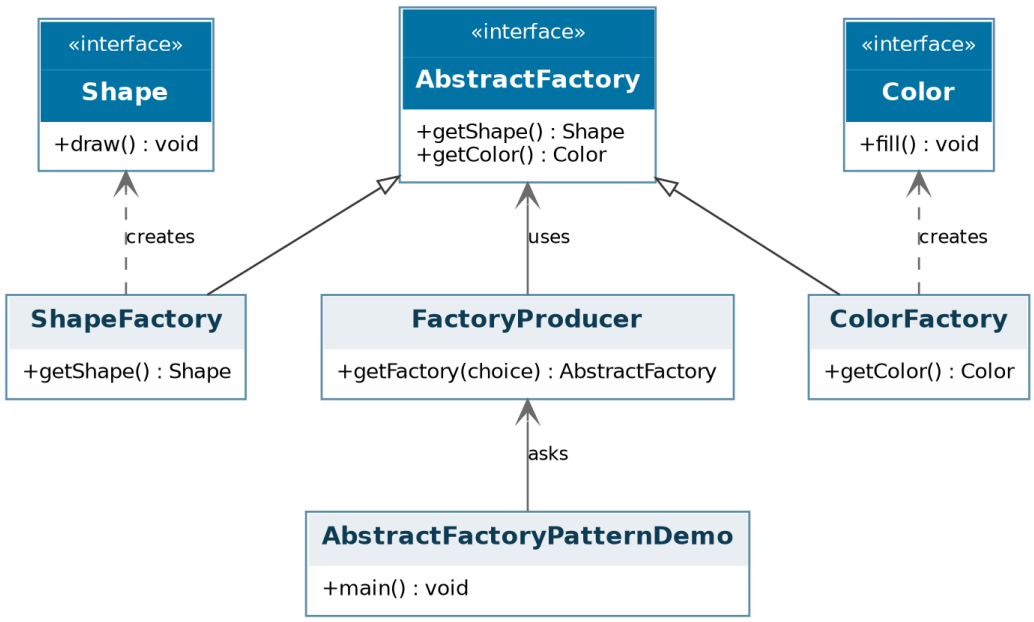
Abstract Factory patterns work around a super-factory which creates other factories. In the Abstract Factory pattern, an interface is responsible for creating a factory of related objects without explicitly specifying their classes. Each generated factory can give objects as per the Factory pattern.

> **Real-world analogy:** A furniture store that sells matched sets: choose "Victorian" or "Modern" and you automatically get a chair, sofa, and table that are all styled consistently — you never mix a Modern chair with a Victorian sofa by accident.

When to reach for it

- A system needs to work with multiple families of related products (e.g. Shape family and Color family).
- You want to enforce that products from one family are always used together.
- You want to swap an entire family of objects at runtime by swapping one factory.

UML structure



UML Element	Abstract Factory Pattern
AbstractFactory «interface»	Declares getShape(): Shape and getColor(): Color.
ShapeFactory / ColorFactory	Concrete factories extending AbstractFactory; each creates one family of related objects.
FactoryProducer	Static getFactory(String choice) returns the right concrete AbstractFactory ("super-factory of factories").
Shape / Color «interfaces»	Each has concrete implementations (Circle/Square/Rectangle, Red/Green/Blue).

Hands-on: Java implementation

```
public class AbstractFactoryPatternDemo {
  public static void main(String[] args) {

    //get shape factory
    AbstractFactory shapeFactory = FactoryProducer.getFactory("SHAPE");

    //get an object of Shape Circle
    Shape shape1 = shapeFactory.getShape("CIRCLE");
    shape1.draw();

    //get color factory
    AbstractFactory colorFactory = FactoryProducer.getFactory("COLOR");

    //get an object of Color Red
    Color color1 = colorFactory.getColor("RED");
    color1.fill();
  }
}

public class FactoryProducer {
  public static AbstractFactory getFactory(String choice){
    if(choice.equalsIgnoreCase("SHAPE")){
      return new ShapeFactory();
    } else if(choice.equalsIgnoreCase("COLOR")){
      return new ColorFactory();
    }
    return null;
  }
}
```

Walking through the code

- `FactoryProducer` is the entry point — the client asks it for a "SHAPE" or "COLOR" factory.
- Each returned factory (ShapeFactory, ColorFactory) is itself a Factory pattern underneath.
- The client always programs against `AbstractFactory`, `Shape`, and `Color` — never a concrete class name.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Guarantees products from the same family are used together. • Isolates concrete classes from the client entirely. • Swapping an entire product family = swapping one factory instance.	• Adding a brand-new kind of product to every family means touching every concrete factory. • More classes and interfaces than a plain Factory — overkill for a single product family.

2.3 Singleton Pattern

> *Ensure a class has only one instance, and provide a global point of access to it.*

Definition

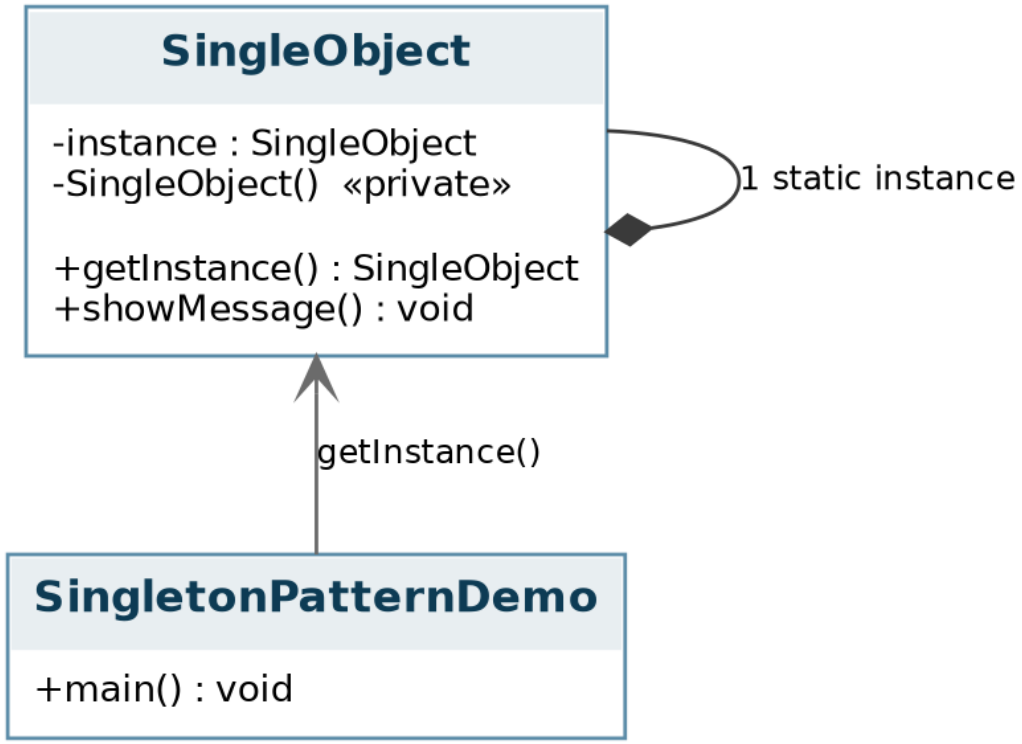
This pattern involves a single class which is responsible for creating an object while making sure that only a single object gets created. Ensure a class has only one instance, and provide a global point of access to it.

> **Real-world analogy:** A country has exactly one government office that issues passports. You don't create your own passport office — you go to the one that already exists.

When to reach for it

- Exactly one instance of a class is needed to coordinate actions across the system (e.g. a configuration manager, connection pool, or logger).
- You need stricter control over global state than a plain global variable offers.

UML structure



UML Element	Singleton Pattern
SingleObject	Holds a private static instance of itself, created once.
Constructor	Made private so the class cannot be instantiated from outside.
getInstance()	Public static method that returns the single shared instance.

Hands-on: Java implementation

```
public class SingleObject {

  //create an object of SingleObject
  private static SingleObject instance = new SingleObject();

  //make the constructor private so that this class cannot be
  //instantiated
  private SingleObject(){}
```

```
  //Get the only object available
  public static SingleObject getInstance(){
    return instance;
  }

  public void showMessage(){
    System.out.println("Hello World!");
  }
}

public class SingletonPatternDemo {
  public static void main(String[] args) {

    //illegal construct
    //Compile Time Error: The constructor SingleObject() is not visible
    //SingleObject object = new SingleObject();

    //Get the only object available
    SingleObject object = SingleObject.getInstance();

    //show the message
    object.showMessage();
  }
}
```

Walking through the code

- The private constructor blocks `new SingleObject()` from outside the class — that line is a compile error, as the comment shows.
- The single instance is created once, eagerly, when the class is loaded.
- Every caller reaches the same object through `getInstance()`.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Guarantees a single, globally reachable instance. • Lazy or eager initialization control in one place.	• Introduces global state, which can make unit testing harder. • Can hide dependencies — classes reach out to a singleton instead of receiving it, which weakens dependency injection. • Needs extra care (synchronization) in multi-threaded environments if lazily initialized.

2.4 Builder Pattern

> *Build a complex object using simple objects and a step-by-step approach.*

Definition

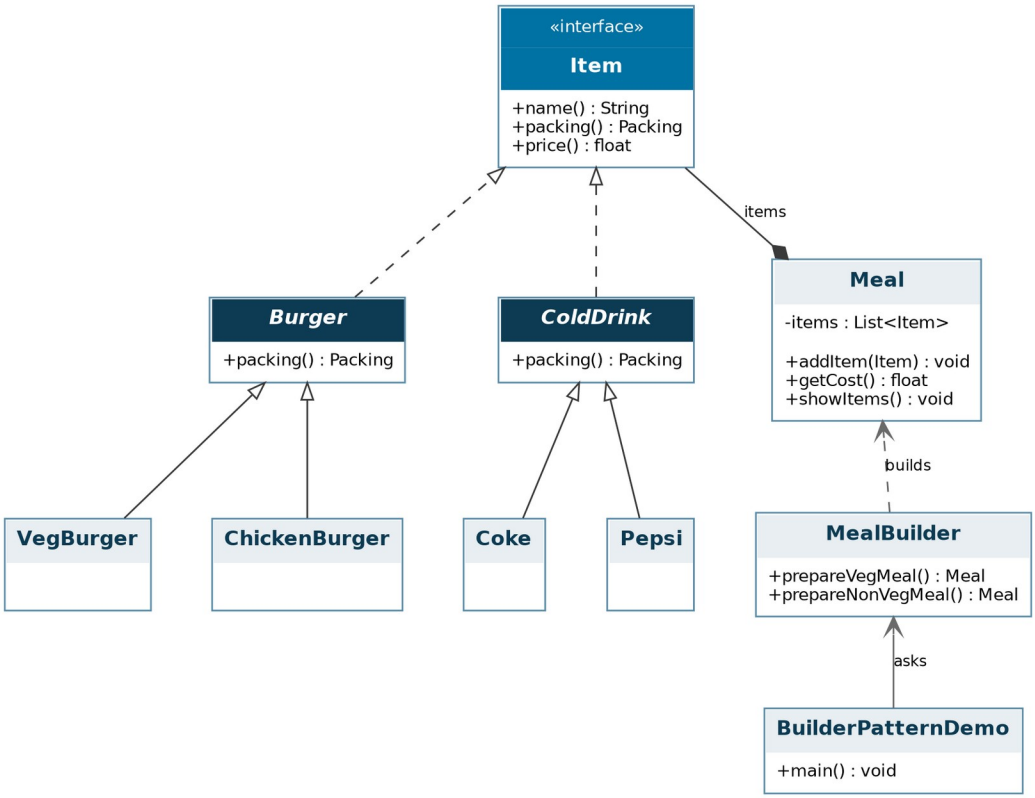
Builder pattern builds a complex object using simple objects and using a step-by-step approach.

> **Real-world analogy:** Ordering a combo meal: instead of assembling a burger, drink, and wrapper yourself, you tell the counter "veg meal" or "non-veg meal" and a MealBuilder assembles the right combination of Items step by step.

When to reach for it

- An object requires many optional parts or a specific construction sequence.
- You want the same construction process to produce different representations (a veg meal vs a non-veg meal).
- You want to keep the final object immutable once built.

UML structure



UML Element	Builder Pattern
Item «interface»	Declares name(), packing(): Packing, price(): float.
Burger / ColdDrink	Abstract item families; VegBurger, ChickenBurger, Coke, Pepsi extend them.
Packing «interface»	Wrapper / Bottle implement packaging behavior for each item.
Meal	Holds a list of Items; addItem(), getCost(), showItems().
MealBuilder	prepareVegMeal(): Meal and prepareNonVegMeal(): Meal assemble a Meal step by step.

Hands-on: Java implementation

```
public class MealBuilder {

  public Meal prepareVegMeal (){
    Meal meal = new Meal();
```

```
    meal.addItem(new VegBurger());
    meal.addItem(new Coke());
    return meal;
  }

  public Meal prepareNonVegMeal (){
    Meal meal = new Meal();
    meal.addItem(new ChickenBurger());
    meal.addItem(new Pepsi());
    return meal;
  }
}

public class BuilderPatternDemo {
  public static void main(String[] args) {

    MealBuilder mealBuilder = new MealBuilder();

    Meal vegMeal = mealBuilder.prepareVegMeal();
    System.out.println("Veg Meal");
    vegMeal.showItems();
    System.out.println("Total Cost: " + vegMeal.getCost());

    Meal nonVegMeal = mealBuilder.prepareNonVegMeal();
    System.out.println("\n\nNon-Veg Meal");
    nonVegMeal.showItems();
    System.out.println("Total Cost: " + nonVegMeal.getCost());
  }
}
```

Walking through the code

- `MealBuilder` hides the multi-step assembly of a `Meal` behind one call: `prepareVegMeal()`.
- Each `Item` (VegBurger, Coke…) knows its own name, packing, and price — `Meal` just aggregates them.
- The client only ever deals with the finished `Meal`, never assembles items manually.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Separates the construction of a complex object from its representation. • Same building process can yield different finished objects. • Improves readability versus a constructor with a dozen parameters.	• Requires creating a separate Builder class per product family. • Overkill for objects that are simple to construct.

2.5 Prototype Pattern

> *When creating an object is costly and you already have a similar one, clone it instead.*

Definition

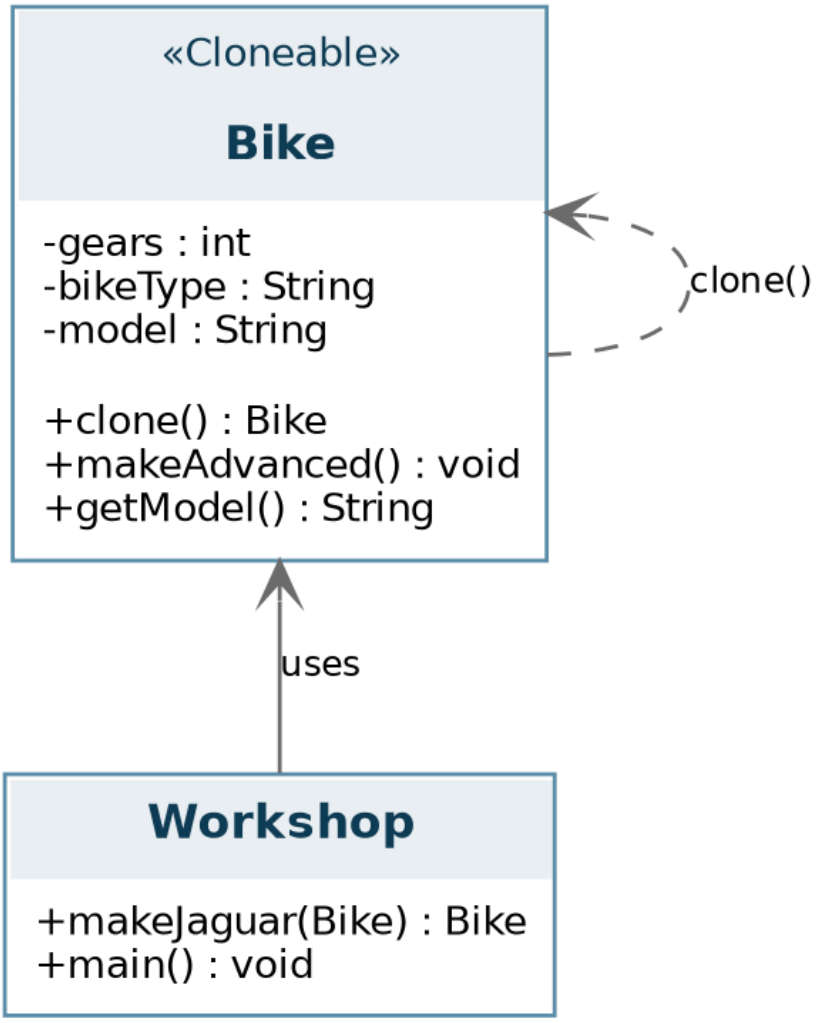
When creating an object is time consuming and a costly affair, and you already have a most similar object instance in hand, then go for the Prototype pattern. Instead of going through a time-consuming process to create a complex object, just copy the existing similar object and modify it according to your needs.

> **Real-world analogy:** A workshop that turns a standard bike into a custom Jaguar model doesn't build the bike from raw metal each time — it clones an existing basic Bike and then upgrades the clone.

When to reach for it

- Object creation is expensive (heavy computation, I/O, network calls) compared to copying.
- You need many objects that are only slightly different from an existing one.
- You want to avoid subclassing just to get object variants.

UML structure



UML Element	Prototype Pattern
Bike implements Cloneable	Holds gears, bikeType, model; provides clone(): Bike and makeAdvanced().
clone()	Returns a new Bike copy instead of re-running full construction logic.
Workshop	Client-side helper; makeJaguar(Bike) takes a cloned bike and advances it.

Hands-on: Java implementation

```
class Bike implements Cloneable {
  private int gears;
  private String bikeType;
```

```
  private String model;
  public Bike() {
    bikeType = "Standard";
    model = "Leopard";
    gears = 4;
  }

  public Bike clone() {
    return new Bike();
  }

  public void makeAdvanced() {
    bikeType = "Advanced";
    model = "Jaguar";
    gears = 6;
  }
  public String getModel(){
    return model;
  }
}

public class Workshop {
  public Bike makeJaguar(Bike basicBike) {
    basicBike.makeAdvanced();
    return basicBike;
  }
  public static void main(String args[]){
    Bike bike = new Bike();
    Bike basicBike = bike.clone();
    Workshop workShop = new Workshop();
    Bike advancedBike = workShop.makeJaguar(basicBike);
    System.out.println("Prototype Design Pattern: "+advancedBike.getModel());
  }
}
```

Walking through the code

- `bike.clone()` produces a fresh `Bike` without repeating whatever heavy setup the constructor represents.
- `Workshop.makeJaguar()` then customizes the clone — the original `bike` object is left untouched.
- In real systems, `clone()` would deep-copy expensive internal state instead of just calling `new Bike()` again.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Avoids repeating expensive initialization for near-identical objects. • Adds/removes products at runtime without a class hierarchy explosion.	• Cloning objects with complex or circular references can be tricky (deep vs shallow copy). • Every prototype class must implement its own cloning logic correctly.

3. Structural Patterns

Structural patterns concern class and object composition. The concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities — the goal is to keep structures flexible and efficient without duplicating code.

3.1 Adapter Pattern

> *Convert the interface of a class into another interface clients expect, letting incompatible classes work together.*

Definition

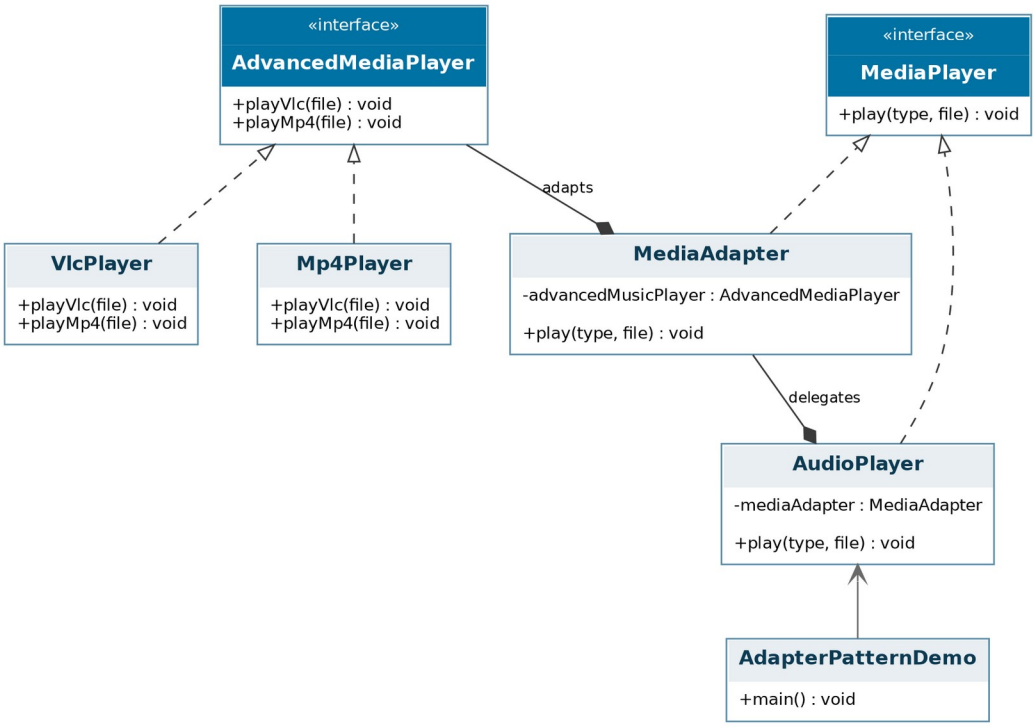
The Adapter pattern acts as a bridge between two incompatible interfaces. It wraps an existing class with a new interface so that classes with incompatible interfaces can work together without modifying their existing source code. This pattern involves a single class — the adapter — which is responsible for joining functionalities of independent or incompatible interfaces.

> **Real-world analogy:** A travel power plug adapter: your laptop charger (client) expects a certain plug shape, but the wall socket (existing, incompatible interface) is a different shape entirely. The adapter sits between them, translating one shape into the other without changing either the charger or the wall socket.

When to reach for it

- You want to use an existing class, but its interface doesn't match what your code expects.
- You need to integrate a third-party or legacy class whose source you can't (or shouldn't) modify.
- You want to create a reusable class that cooperates with unrelated or unforeseen classes — ones that don't necessarily have compatible interfaces.

UML structure



UML Element	Adapter Pattern
MediaPlayer «interface»	The target interface clients already expect: play(audioType, fileName).
AdvancedMediaPlayer «interface»	The existing, incompatible interface (the "adaptee" family): playVlc(), playMp4().
VlcPlayer / Mp4Player	Concrete implementations of AdvancedMediaPlayer — the classes we can't change.
MediaAdapter	Implements MediaPlayer; internally holds an AdvancedMediaPlayer and translates play() calls into playVlc()/playMp4().
AudioPlayer	The client-facing class; implements MediaPlayer directly for mp3, and delegates everything else to a MediaAdapter.

Hands-on: Java implementation

```
public interface MediaPlayer {
  public void play(String audioType, String fileName);
}

public interface AdvancedMediaPlayer {
  public void playVlc(String fileName);
  public void playMp4(String fileName);
}

public class VlcPlayer implements AdvancedMediaPlayer{
  @Override
  public void playVlc(String fileName) {
    System.out.println("Playing vlc file. Name: "+ fileName);
  }
  @Override
  public void playMp4(String fileName) {
    //do nothing
  }
}

public class Mp4Player implements AdvancedMediaPlayer{
  @Override
  public void playVlc(String fileName) {
    //do nothing
  }
  @Override
  public void playMp4(String fileName) {
    System.out.println("Playing mp4 file. Name: "+ fileName);
  }
}

public class MediaAdapter implements MediaPlayer {

  AdvancedMediaPlayer advancedMusicPlayer;

  public MediaAdapter(String audioType){
    if(audioType.equalsIgnoreCase("vlc") ){
      advancedMusicPlayer = new VlcPlayer();
    } else if (audioType.equalsIgnoreCase("mp4")){
      advancedMusicPlayer = new Mp4Player();
    }
  }

  @Override
  public void play(String audioType, String fileName) {
    if(audioType.equalsIgnoreCase("vlc")){
      advancedMusicPlayer.playVlc(fileName);
```

```
    } else if(audioType.equalsIgnoreCase("mp4")){
      advancedMusicPlayer.playMp4(fileName);
    }
  }
}

public class AudioPlayer implements MediaPlayer {

  MediaAdapter mediaAdapter;

  @Override
  public void play(String audioType, String fileName) {

    //inbuilt support to play mp3 music files
    if(audioType.equalsIgnoreCase("mp3")){
      System.out.println("Playing mp3 file. Name: " + fileName);
    }
    //mediaAdapter provides support to play other file formats
    else if(audioType.equalsIgnoreCase("vlc") || audioType.equalsIgnoreCase("mp4")){
      mediaAdapter = new MediaAdapter(audioType);
      mediaAdapter.play(audioType, fileName);
    } else{
      System.out.println("Invalid media. " + audioType + " format not supported");
    }
  }
}

public class AdapterPatternDemo {
  public static void main(String[] args) {
    AudioPlayer audioPlayer = new AudioPlayer();

    audioPlayer.play("mp3", "beyond the horizon.mp3");
    audioPlayer.play("mp4", "alone.mp4");
    audioPlayer.play("vlc", "far far away.vlc");
    audioPlayer.play("avi", "mind me.avi");
  }
}
```

Walking through the code

- `AudioPlayer` natively understands only mp3 — for anything else, it hands off to a `MediaAdapter`.
- `MediaAdapter` is the actual adapter: it implements the `MediaPlayer` interface the client expects, but internally calls `playVlc()`/`playMp4()` on the incompatible `AdvancedMediaPlayer` classes.
- Neither `VlcPlayer` nor `Mp4Player` had to change at all — the adapter absorbs the mismatch entirely.
- An unsupported format ("avi") simply falls through to an error message, since no adapter path exists for it.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Lets incompatible interfaces work together without modifying existing, tested code. • Keeps the single-responsibility principle intact — interface conversion lives in one adapter class. • Great fit for wrapping legacy code or third-party libraries.	• Adds an extra layer of indirection between client and adaptee. • Overusing adapters can hide too many small interface mismatches, making the overall design harder to trace.

3.2 Bridge Pattern

> *Decouple an abstraction from its implementation so the two can vary independently.*

Definition

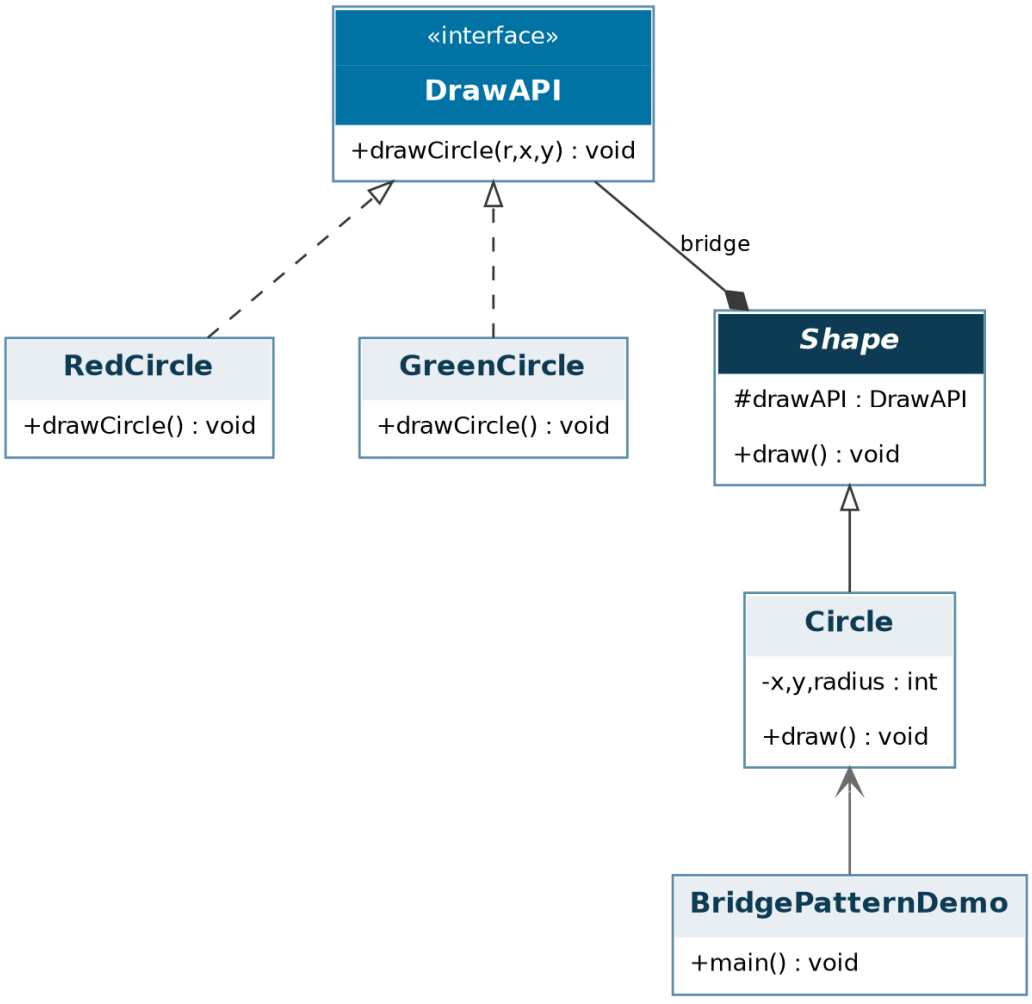
This pattern involves an interface which acts as a bridge, making the functionality of concrete classes independent from interface implementer classes. Both types of classes can be altered structurally without affecting each other. Bridge is used when we need to decouple an abstraction from its implementation so that the two can vary independently.

> **Real-world analogy:** A TV remote (abstraction) and the TV brand it controls (implementation) are separate hierarchies. You can design new remotes or support new TV brands independently, as long as both honor the same signal interface.

When to reach for it

- You want to avoid a permanent binding between an abstraction and its implementation.
- Both the abstraction and implementation should be extensible via subclassing independently.
- Changes in the implementation should have no impact on client code.

UML structure



UML Element	Bridge Pattern
DrawAPI «interface»	Declares drawCircle(radius, x, y). RedCircle and GreenCircle implement it (the "implementation" hierarchy).
Shape (abstract)	Holds a DrawAPI reference; declares abstract draw(). This is the "abstraction" hierarchy.
Circle extends Shape	Delegates the actual drawing to whichever DrawAPI it was given, via the bridge.

Hands-on: Java implementation

```
public interface DrawAPI {
  public void drawCircle(int radius, int x, int y);
}

public abstract class Shape {
  protected DrawAPI drawAPI;

  protected Shape(DrawAPI drawAPI){
    this.drawAPI = drawAPI;
  }
  public abstract void draw();
}

public class Circle extends Shape {
  private int x, y, radius;

  public Circle(int x, int y, int radius, DrawAPI drawAPI) {
    super(drawAPI);
    this.x = x;
    this.y = y;
    this.radius = radius;
  }

  public void draw() {
    drawAPI.drawCircle(radius,x,y);
  }
}
```

Walking through the code

- `Shape` never implements drawing itself — it holds a `DrawAPI` and forwards the call: that reference is the bridge.
- Swapping `RedCircle` for `GreenCircle` changes rendering without touching `Circle` or `Shape` at all.
- New shapes (Square, Rectangle) and new draw implementations can be added on separate, independent hierarchies.

Trade-offs

✔ Advantages	✘ Trade-offs
• Abstraction and implementation can evolve independently. • Avoids a combinatorial explosion of subclasses (e.g. RedCircle, RedSquare, GreenCircle, GreenSquare...).	• Adds indirection that can make the design harder to follow at first glance. • Best justified when you already anticipate variation on both axes — otherwise it's premature abstraction.

3.3 Composite Pattern

> *Treat a group of objects the same way you'd treat a single object.*

Definition

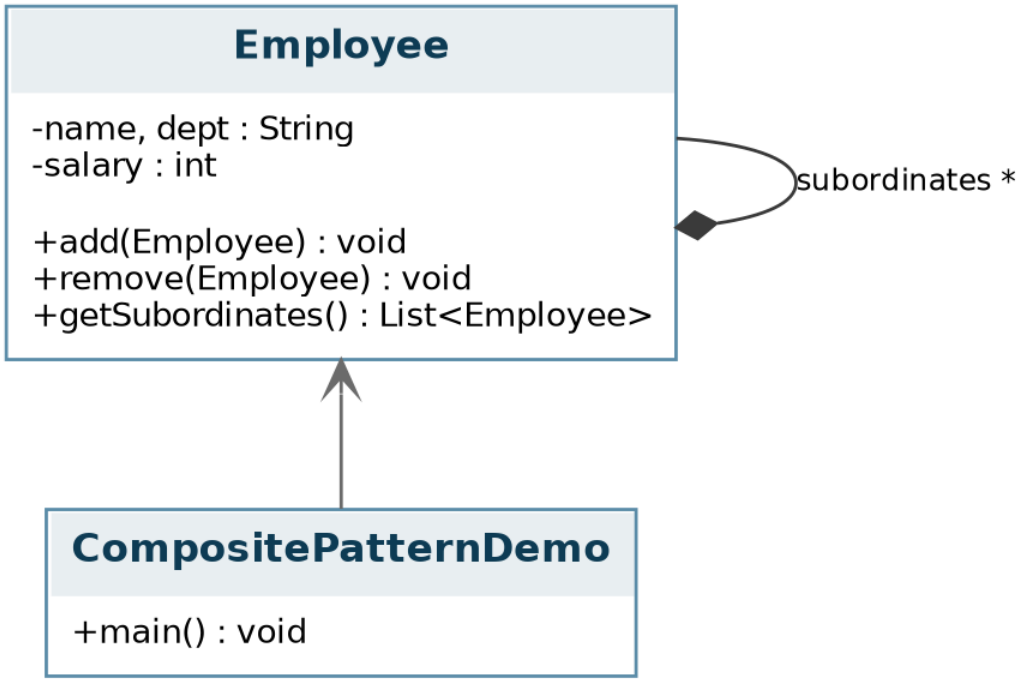
Composite pattern is used where we need to treat a group of objects in a similar way as a single object. It composes objects in terms of a tree structure to represent part-as-well-as-whole hierarchy. This pattern creates a class that contains a group of its own objects, creating a tree structure.

> **Real-world analogy:** An org chart: a CEO manages several department heads, each of whom may manage their own reports. Whether you're looking at one employee or an entire department, you interact with them the same way — as an Employee.

When to reach for it

- You need to represent part-whole hierarchies of objects (trees).
- You want client code to treat individual objects and compositions of objects uniformly.

UML structure



UML Element	Composite Pattern
Employee	Holds name, dept, salary, and a List<Employee> subordinates — the composite is itself the component type.
add(Employee) / remove(Employee)	Manage the tree structure of subordinates.
getSubordinates()	Returns the child nodes, letting client code walk the hierarchy uniformly.

Hands-on: Java implementation

```
public class Employee {
  private String name;
```

```
  private String dept;
  private int salary;
  private List<Employee> subordinates;
  // constructor
  public Employee(String name,String dept, int sal) {
    this.name = name;
    this.dept = dept;
    this.salary = sal;
    subordinates = new ArrayList<Employee>();
  }
  public void add(Employee e) {
    subordinates.add(e);
  }
  public void remove(Employee e) {
    subordinates.remove(e);
  }
  public List<Employee> getSubordinates(){
    return subordinates;
  }
}

public class CompositePatternDemo {
  public static void main(String[] args) {
    Employee CEO = new Employee("John","CEO", 30000);
    Employee headSales = new Employee("Robert","Head Sales", 20000);
    Employee headMarketing = new Employee("Michel","Head Marketing", 20000);
    CEO.add(headSales);
    CEO.add(headMarketing);
  }
}
```

Walking through the code

- Every node in the tree — CEO or an individual sales rep — is an `Employee`.
- `add()`/`remove()` let any Employee node gain children, forming an arbitrarily deep tree.
- Client code that asks an Employee for its subordinates works the same whether it's a leaf or a branch.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Simplifies client code — no need to distinguish leaf vs composite objects. • Makes it easy to add new kinds of components.	• Can make the design overly general — sometimes it's hard to restrict what kinds of children a component may hold.

3.4 Decorator Pattern

> *Add new functionality to an existing object without altering its structure.*

Definition

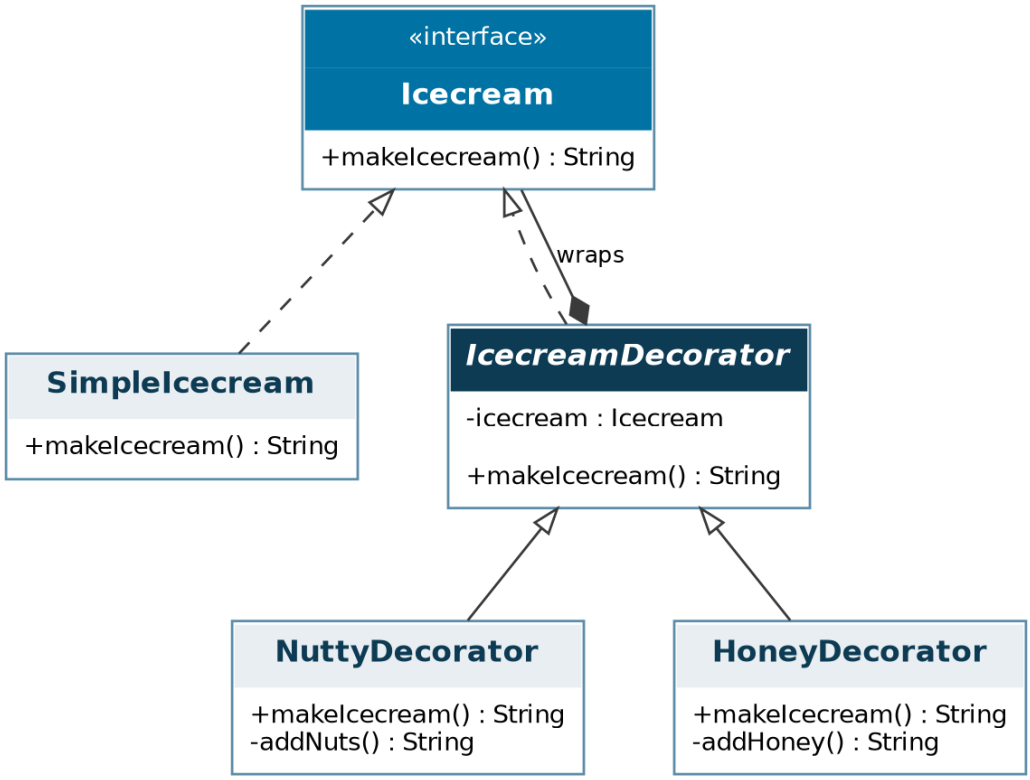
Decorator pattern allows a user to add new functionality to an existing object without altering its structure. This pattern creates a decorator class which wraps the original class and provides additional functionality, keeping the class method's signature intact.

> **Real-world analogy:** Ordering ice cream and asking for nuts and honey on top — the base ice cream doesn't change; toppings wrap around it and add flavor, and you can stack as many as you like in any order.

When to reach for it

- You want to add responsibilities to individual objects dynamically, without affecting other objects of the same class.
- Extension by subclassing is impractical because of the number of independent combinations possible.

UML structure



UML Element	Decorator Pattern
Icecream «interface»	Declares makeIcecream().
SimpleIcecream	Concrete base component implementing Icecream directly.
IcecreamDecorator	Also implements Icecream, but wraps another Icecream instance.
NuttyDecorator / HoneyDecorator	Extend IcecreamDecorator, calling the wrapped object's makeIcecream() then adding their own touch (addNuts()/addHoney()).

Hands-on: Java implementation

```
public class NuttyDecorator extends IcecreamDecorator {

  public NuttyDecorator(Icecream specialIcecream) {
    super(specialIcecream);
  }

  public String makeIcecream() {
    return specialIcecream.makeIcecream() + addNuts();
  }
```

```
  private String addNuts() {
    return " + cruncy nuts";
  }
}
```

Walking through the code

- `NuttyDecorator` wraps another `Icecream` (`specialIcecream`) rather than replacing it.
- `makeIcecream()` first calls the wrapped object's own method, then appends its own behavior — decorators chain.
- Wrapping a `SimpleIcecream` in both `NuttyDecorator` and `HoneyDecorator` stacks both toppings without touching either class.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Adds behavior at runtime without subclass explosion. • Follows the Open/Closed Principle — classes are open for extension, closed for modification.	• Many small wrapper classes can make a system harder to understand. • Order of decoration can matter and may be easy to get wrong.

3.5 Facade Pattern

> *Hide the complexity of a subsystem behind one simple interface.*

Definition

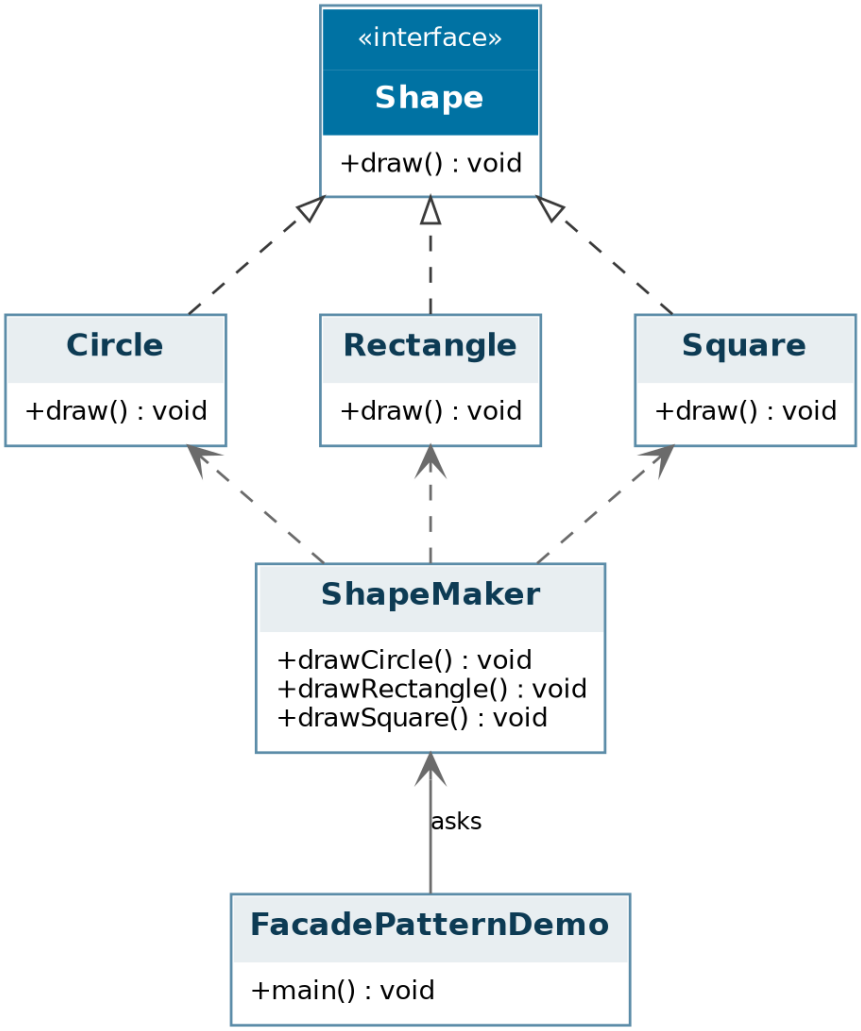
Facade pattern hides the complexities of the system and provides an interface to the client using which the client can access the system. This pattern involves a single class which provides simplified methods required by the client and delegates calls to methods of existing system classes.

> **Real-world analogy:** A car's ignition button: press it, and dozens of subsystems (fuel injection, battery, starter motor) coordinate behind the scenes. You never touch them directly.

When to reach for it

- A subsystem is complex, and most clients only need a simple, common set of operations.
- You want to reduce coupling between client code and a subsystem's internal classes.

UML structure



UML Element	Facade Pattern
ShapeMaker	The facade. Holds Circle, Rectangle, Square internally.
drawCircle()/ drawRectangle()/drawSquare()	Simple methods that internally delegate to the right shape's draw().
FacadePatternDemo	Client; talks only to ShapeMaker, never to Circle/Rectangle/Square directly.

Hands-on: Java implementation

```
public class ShapeMaker {
  private Shape circle;
  private Shape rectangle;
  private Shape square;

  public ShapeMaker() {
    circle = new Circle();
    rectangle = new Rectangle();
    square = new Square();
  }

  public void drawCircle(){
    circle.draw();
  }
```

```
  public void drawRectangle(){
    rectangle.draw();
  }
  public void drawSquare(){
    square.draw();
  }
}

public class FacadePatternDemo {
  public static void main(String[] args) {
    ShapeMaker shapeMaker = new ShapeMaker();

    shapeMaker.drawCircle();
    shapeMaker.drawRectangle();
    shapeMaker.drawSquare();
  }
}
```

Walking through the code

- `ShapeMaker` owns instances of Circle, Rectangle, and Square, hiding their construction from the client.
- The client calls three simple methods instead of managing three separate shape objects itself.
- If the underlying shapes' constructors or APIs change, only `ShapeMaker` needs updating.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Shields clients from subsystem complexity. • Reduces coupling between client code and subsystem internals.	• Can become a "god object" if it grows to expose too much. • Doesn't prevent clients from bypassing it and accessing the subsystem directly if needed.

3.6 Flyweight Pattern

> *Reduce the number of objects created to decrease memory footprint and increase performance.*

Definition

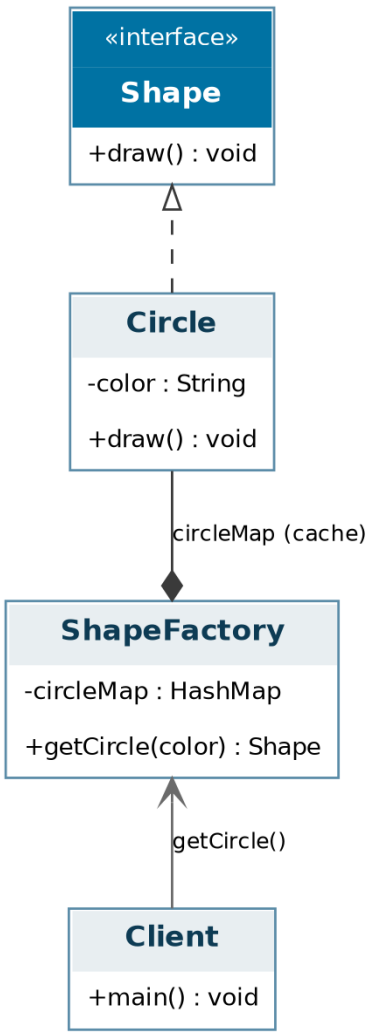
Flyweight pattern is primarily used to reduce the number of objects created and to decrease memory footprint and increase performance. It is used when it is needed to create a large number of objects, because with large numbers memory cost becomes a constraint.

> **Real-world analogy:** A print shop keeps one physical stencil per letter of the alphabet and reuses it for every document, rather than carving a brand-new stencil for every single letter typed.

When to reach for it

- An application needs a huge number of similar objects.
- Most of an object's state can be made extrinsic (passed in) rather than stored per-instance.
- Object storage costs are a real constraint.

UML structure



UML Element	Flyweight Pattern
ShapeFactory	Keeps a HashMap<String, Shape> cache (circleMap).
getCircle(color)	Returns a cached Circle if one already exists for that color; only creates a new one on a cache miss.

Hands-on: Java implementation

```
import java.util.HashMap;

public class ShapeFactory {
  private static final HashMap<String, Shape> circleMap = new HashMap();

  public static Shape getCircle(String color) {
    Circle circle = (Circle)circleMap.get(color);

    if(circle == null) {
      circle = new Circle(color);
      circleMap.put(color, circle);
      System.out.println("Creating circle of color : " + color);
    }
    return circle;
  }
```

```
}
```

Walking through the code

- `circleMap` is keyed by color — the shared, cacheable part of a Circle's state.
- A new `Circle` is only constructed the first time a given color is requested; every later request reuses it.
- Position, size, or other per-use data would normally be passed into draw() as extrinsic state, kept out of the cached object.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Dramatically cuts memory usage when many objects share common state. • Improves performance by avoiding repeated construction.	• Adds complexity in separating intrinsic (shared) from extrinsic (per-use) state. • Shared objects must be immutable or carefully synchronized if mutable.

3.7 Proxy Pattern

> *Provide a stand-in object that controls access to another object.*

Definition

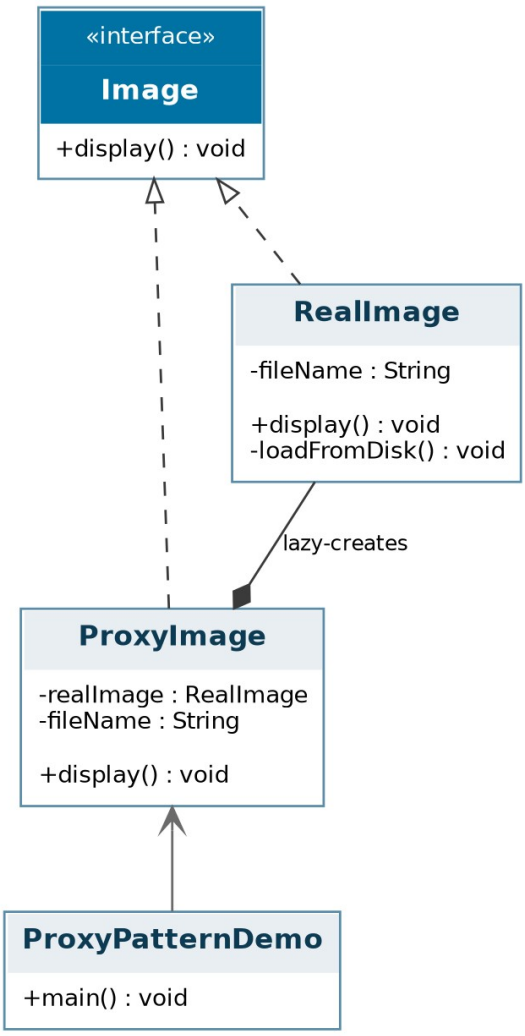
In the Proxy pattern, we create an object having the original object, to interface its functionality to the outer world.

> **Real-world analogy:** A receptionist who screens visitors before they reach an executive — the visitor thinks they're dealing directly with the executive, but the receptionist controls access (and may not even connect the call at all).

When to reach for it

- You want to defer the cost of creating an expensive object until it's actually needed (virtual proxy).
- You need to control or restrict access to an object (protection proxy).
- You want to add logging, caching, or lazy loading transparently.

UML structure



UML Element	Proxy Pattern
Image «interface»	Declares display().
RealImage	The real, expensive object — loadFromDisk() happens in its constructor.
ProxyImage	Implements Image; holds a RealImage reference that starts null and is only created on first display().

Hands-on: Java implementation

```
public class ProxyImage implements Image{

  private RealImage realImage;
  private String fileName;

  public ProxyImage(String fileName){
    this.fileName = fileName;
  }

  @Override
  public void display() {
    if(realImage == null){
      realImage = new RealImage(fileName);
    }
```

```
    realImage.display();
  }
}

public class ProxyPatternDemo {

  public static void main(String[] args) {
    Image image = new ProxyImage("test_10mb.jpg");

    //image will be loaded from disk
    image.display();
    System.out.println("");

    //image will not be loaded from disk
    image.display();
  }
}
```

Walking through the code

- `ProxyImage` looks like a real `Image` to the client, but doesn't create the expensive `RealImage` until it's actually needed.
- The first `display()` call triggers `loadFromDisk()`; every subsequent call reuses the already-loaded `RealImage`.
- The client never notices the difference — it just calls `display()` on the `Image` interface.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Enables lazy initialization, access control, logging, or caching transparently. • Client code is unaware whether it's talking to the real object or its proxy.	• Adds another layer of indirection, which can add latency or complexity. • Easy to accidentally leak proxy-specific behavior into what should be a transparent substitution.

4. Behavioral Patterns

Behavioral patterns encourage loose coupling between interacting objects. They focus on how responsibilities are distributed and how objects communicate, rather than on how they're structured or created. The important behavioral patterns covered here are Chain of Responsibility, Command, Iterator, and Template Method.

4.1 Chain of Responsibility Pattern

> *Pass a request along a chain of handlers until one of them handles it.*

Definition

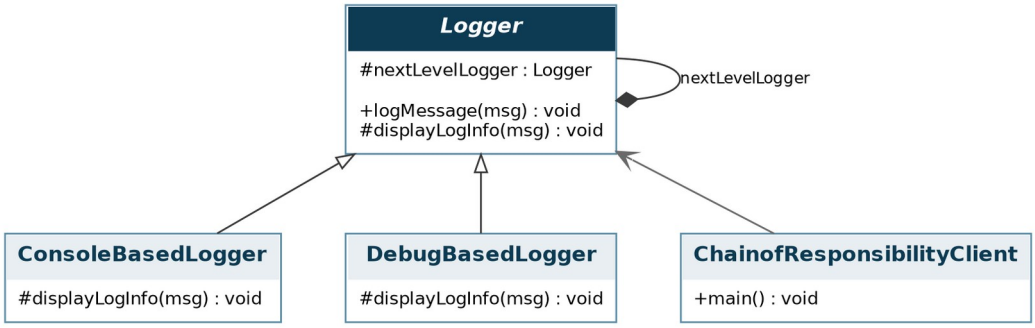
It is used when the processing has to be sequential and the processing applied needs to change dynamically. All the processors in the chain implement a common interface. A processor has a reference to the next processor in the chain. A processor can decide for itself whether it needs to process the input and whether it needs to call the next processor in the chain. A classic real-world example is Servlet Filters.

> **Real-world analogy:** A support ticket that goes from a first-line agent, to a specialist, to a manager — each person along the chain either resolves it or passes it further up, and the customer never needs to know how many people were involved.

When to reach for it

- More than one object may handle a request, and the handler isn't known in advance.
- You want to issue a request without specifying the receiver explicitly.
- The set of handlers and their order should be configurable at runtime.

UML structure



UML Element	Chain of Responsibility
Logger (abstract)	Holds nextLevelLogger; declares logMessage(msg) and abstract displayLogInfo(msg).
ConsoleBasedLogger / DebugBasedLogger	Concrete handlers; each implements its own displayLogInfo().
setNextLevelLogger()	Wires one handler to the next, forming the chain.

Hands-on: Java implementation

```
public abstract class Logger {
  protected Logger nextLevelLogger;

  public void setNextLevelLogger(Logger nextLevelLogger) {
```

```
    this.nextLevelLogger = nextLevelLogger;
  }
  public void logMessage(String msg){
    displayLogInfo(msg);
    if (nextLevelLogger!=null) {
      nextLevelLogger.logMessage(msg);
    }
  }
  protected abstract void displayLogInfo(String msg);
}

public class ConsoleBasedLogger extends Logger {
  @Override
  protected void displayLogInfo(String msg) {
    System.out.println("CONSOLE LOGGER INFO: "+msg);
  }
}

public class DebugBasedLogger extends Logger {
  @Override
  protected void displayLogInfo(String msg) {
    System.out.println("DEBUG LOGGER INFO: "+msg);
  }
}

public class ChainofResponsibilityClient {
  private static Logger doChaining(){
    Logger consoleLogger = new ConsoleBasedLogger();
    Logger debugLogger = new DebugBasedLogger();
    consoleLogger.setNextLevelLogger(debugLogger);
    return consoleLogger;
  }
  public static void main(String args[]){
    Logger chainLogger= doChaining();

    chainLogger.logMessage( "Enter the sequence of values ");
    chainLogger.logMessage( "An error is occured now");
    chainLogger.logMessage( "This was the error now debugging is compeled");
  }
}
```

Walking through the code

- `logMessage()` always runs the current handler's own logic (`displayLogInfo`) first.
- It then forwards the same message to `nextLevelLogger` — if one is wired up.
- The client (`ChainofResponsibilityClient`) only talks to the head of the chain; it has no idea how many loggers actually run.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Decouples sender from receiver — the client doesn't need to know which handler will act. • Chain order and membership can be changed at runtime without touching client code.	• No guarantee a request will be handled at all if no handler is a match. • Long chains can be harder to debug and reason about performance-wise.

4.2 Command Pattern

> *Encapsulate a request as an object, so it can be queued, logged, or undone.*

Definition

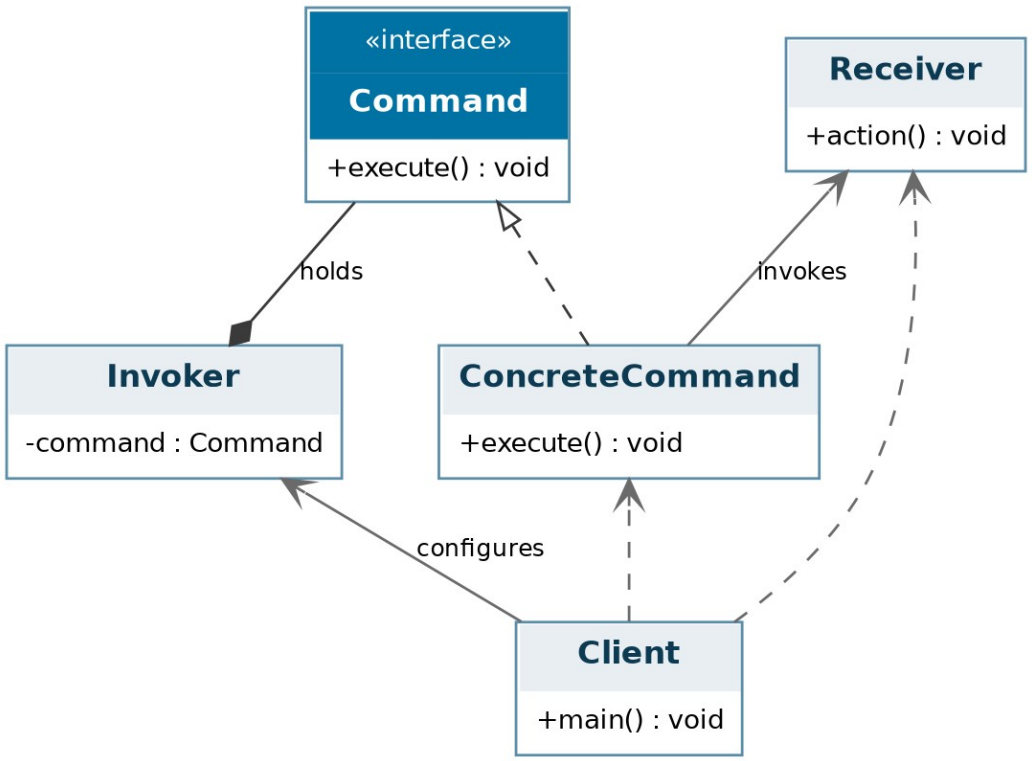
The command pattern is a design pattern that enables all of the information for a request to be contained within a single object. The command can then be invoked as required, often as part of a batch of queued commands with rollback capabilities. It is useful when supporting activities that require the execution of a series of commands. Command objects can be held in a queue and processed sequentially; if each command is stored on a stack after execution and the commands are reversible, this enables a rollback or multi-level undo facility.

> **Real-world analogy:** A restaurant order slip: the waiter (invoker) doesn't cook — they just carry the slip (command object) to the kitchen (receiver), which knows how to execute it. The same slip could be filed for later, repeated, or cancelled.

When to reach for it

- A history of requests is needed.
- You need callback-style functionality.
- Requests need to be handled at different times or in different orders.
- The invoker should be decoupled from the object that actually handles the request.

UML structure



UML Element	Command Pattern
Command «interface»	Declares execute().
ConcreteCommand	Implements execute() by delegating to a Receiver's action().
Receiver	Contains the real implementation of the desired action().
Invoker	Holds a Command and triggers execute() without knowing what it actually

UML Element	Command Pattern
	does.
Client	Creates the Receiver, wraps it in a ConcreteCommand, hands the command to the Invoker.

Hands-on: Java implementation

```
// Command interface — every concrete command implements this
public interface Command {
  void execute();
}

// Receiver — knows HOW to actually perform the work
public class Light {
  private String location;

  public Light(String location) {
    this.location = location;
  }

  public void on() {
    System.out.println(location + " light is ON");
  }

  public void off() {
    System.out.println(location + " light is OFF");
  }
}

// ConcreteCommand — binds a Receiver to a specific action
public class LightOnCommand implements Command {
  private Light light;

  public LightOnCommand(Light light) {
    this.light = light;
  }

  @Override
  public void execute() {
    light.on();
  }
}

public class LightOffCommand implements Command {
  private Light light;

  public LightOffCommand(Light light) {
    this.light = light;
  }

  @Override
  public void execute() {
    light.off();
  }
}

// Invoker — stores a command and fires it later, without knowing what it does
public class RemoteControl {
  private Command slot;

  public void setCommand(Command command) {
    this.slot = command;
```

```
  }

  public void pressButton() {
    slot.execute();
  }
}

// Client — wires Receiver -> ConcreteCommand -> Invoker
public class CommandPatternDemo {
  public static void main(String[] args) {
    Light livingRoomLight = new Light("Living Room");

    Command lightOn = new LightOnCommand(livingRoomLight);
    Command lightOff = new LightOffCommand(livingRoomLight);

    RemoteControl remote = new RemoteControl();

    remote.setCommand(lightOn);
    remote.pressButton();   // Living Room light is ON

    remote.setCommand(lightOff);
    remote.pressButton();   // Living Room light is OFF
  }
}
```

Walking through the code

- Unlike Chain of Responsibility, where each step in a sequence is executed automatically, in Command the Invoker explicitly decides when to execute the command(s) it holds.
- `Light` is the Receiver — it contains the real implementation of on()/off(), with no idea a "command" concept even exists.
- `LightOnCommand` and `LightOffCommand` are ConcreteCommands — each just binds a Receiver reference to one specific action and exposes it uniformly through `execute()`.
- `RemoteControl` is the Invoker — it holds whatever `Command` was last plugged into it via `setCommand()`, and blindly calls `execute()` on `pressButton()`. It never mentions `Light` at all.
- The `CommandPatternDemo` client is the only place that knows the full wiring: it creates the `Light`, wraps it in commands, and loads those commands into the remote.
- Because each request is now an object, this same shape naturally extends to queuing commands, logging them, or pushing them onto a stack to support undo/redo.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Decouples the object that invokes an operation from the one that performs it. • Supports undo/redo, queuing, and logging of requests naturally.	• Introduces a Command class for every distinct action, which can multiply classes quickly.

4.3 Iterator Pattern

> *Access the elements of a collection sequentially without exposing its internal structure.*

Definition

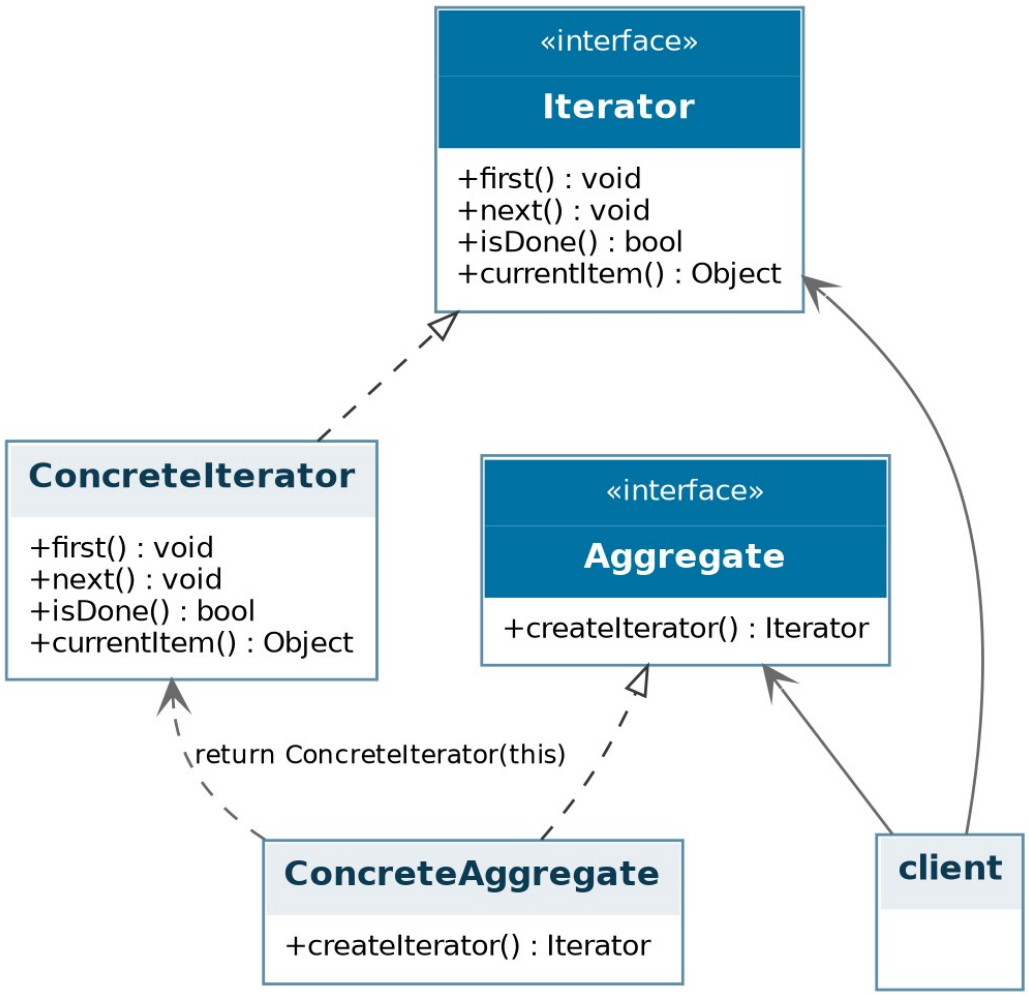
Iterator Pattern provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation. Also known as Cursor: when using an Iterator, we relieve the aggregate of the responsibility of supporting operations for traversing its data, and the Iterator provides a common interface for traversing the items, allowing polymorphic iteration.

> **Real-world analogy:** A TV remote's channel-up button: you don't need to know how channels are stored internally (an array, a list, a broadcast table) — you just press "next" and get the next one.

When to reach for it

- You need to access an aggregate's contents without exposing its internal representation.
- You need to support multiple simultaneous traversals of the same aggregate.
- You want one uniform interface for traversing different aggregate structures (polymorphic iteration).

UML structure



UML Element	Iterator Pattern
Aggregate «interface»	Declares createIterator().
ConcreteAggregate	Implements createIterator(), returning `new ConcreteIterator(this)`.
Iterator «interface»	Declares first(), next(), isDone(), currentItem().
ConcreteIterator	Implements traversal logic specific to the ConcreteAggregate's internal

UML Element	Iterator Pattern
	storage.

Walking through the code

- The client asks a `ConcreteAggregate` for `createIterator()` rather than reaching into its internal data structure.
- The returned `ConcreteIterator` exposes `first()`, `next()`, `isDone()`, and `currentItem()` — a uniform contract regardless of what's being iterated.
- Because iteration logic lives in the Iterator rather than the Aggregate, multiple independent traversals of the same collection can run concurrently.

Trade-offs

✔️ Advantages	✘ Trade-offs
• Simplifies the aggregate's interface by moving traversal logic out. • Supports multiple, independent traversals of the same collection. • Enables polymorphic iteration across very different aggregate structures.	• For simple collections, a dedicated Iterator class can be unnecessary overhead versus using built-in language iteration.

4.4 Template Method Pattern

> *Define the skeleton of an algorithm, letting subclasses fill in specific steps.*

Definition

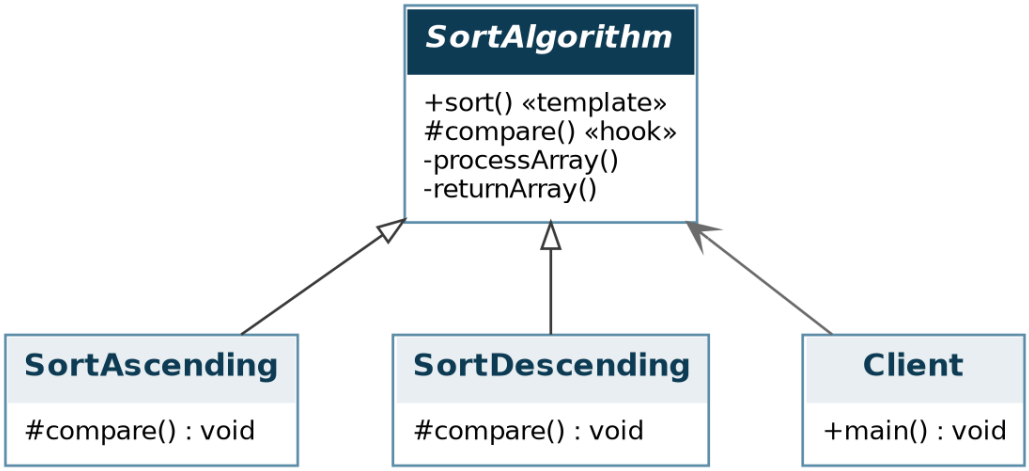
Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure. It should be used to implement the invariant parts of an algorithm once, leaving subclasses to implement the behavior that varies; when common behavior among subclasses should be factored and localized to avoid duplication; and to control subclass extension by exposing "hook" operations at specific points.

> **Real-world analogy:** A recipe card that says "1) boil water, 2) add your chosen flavor packet, 3) simmer, 4) serve" — the overall sequence never changes, but step 2 is a hook that each variant (chicken, tomato, miso) fills in differently.

When to reach for it

- Several subclasses share the same overall algorithm but differ in specific steps.
- You want to prevent subclasses from altering the algorithm's overall structure (declare the template method `final`).
- You want to factor out common behavior into one class to avoid duplicating it across subclasses.

UML structure



UML Element	Template Method Pattern
SortAlgorithm (abstract)	sort() is the template method; calls processArray(), compare(), returnArray() in a fixed sequence.
compare() (abstract/hook)	Deferred to subclasses — the one step allowed to vary.
SortAscending / SortDescending	Concrete subclasses that each override only compare(), reusing the rest of sort() unchanged.

Walking through the code

- `sort()` is declared once in the abstract class and is never overridden — it fixes the overall algorithm shape.
- `compare()` is the one "hook" step deferred to subclasses; `SortAscending` and `SortDescending` differ only here.
- Hooks are methods with default or empty behavior in the abstract class that subclasses may override — this is what lets the skeleton stay fixed while specific steps vary.

Trade-offs

✔ Advantages	✘ Trade-offs
• Removes duplicated boilerplate across subclasses that share an algorithm shape. • Gives the base class control over the sequence, preventing subclasses from breaking the overall flow.	• Relies on inheritance, which is a tighter coupling than composition-based alternatives like Strategy. • Can be harder to follow because the algorithm's steps are spread across a base class and several subclasses.

> **Related patterns worth noting:** The Strategy and Template Method patterns both encapsulate algorithms — one by composition (Strategy), one by inheritance (Template Method). The Factory Method pattern is itself considered a specialization of Template Method, applied specifically to object creation.

5. References

URLs

- https://sourcemaking.com/design_patterns
- http://www.oodesign.com/
- http://javapapers.com/category/design-patterns/

Books

- Head First Design Patterns
- Design Patterns: Elements of Reusable Object-Oriented Software — Gamma, Helm, Johnson, Vlissides (the "Gang of Four")

Source deck: Capgemini "Design Patterns" (20140926_iCON PMC.pptx) — content preserved and elaborated for depth and readability.